

■ Jenkins

Complete Interview Preparation Guide

35+ Questions with Clear Explanations & Examples

CI/CD

DevOps

Automation

Pipelines

Plugins

■ Beginner

■ Intermediate

■ Advanced

SECTION
1

Jenkins Basics

Q1 What is Jenkins?

■ Beginner

Jenkins is a free, open-source automation server written in Java. It helps developers automatically build, test, and deploy their software — this is called Continuous Integration and Continuous Delivery (CI/CD).

Think of Jenkins as a **robot assistant** that watches your code repository. Whenever a developer pushes new code, Jenkins automatically picks it up, compiles it, runs tests, and tells you if something broke.

- Free & open-source — no license cost
- Has 1800+ plugins — works with almost every tool
- Runs on Windows, Linux, macOS
- Easy to install — just a .war file
- Large community support

■ Tip: Jenkins is the most popular CI/CD tool in the world. It is used by Netflix, LinkedIn, and thousands of companies.

Q2 What are the main features of Jenkins?

■ Beginner

Jenkins comes packed with features that make it the go-to tool for automation:

- **Free & Open Source** — No licensing fees. The source code is available on GitHub.
- **Plugin Ecosystem** — Over 1800+ plugins to connect with Git, Maven, Docker, AWS, Slack, etc.
- **Easy Installation** — Just run `java -jar jenkins.war` and it's up in minutes.
- **Cross-Platform** — Runs on any OS that supports Java.
- **Pipeline Support** — Write build/test/deploy steps as code (Jenkinsfile).
- **Distributed Builds** — Master-Agent architecture to run jobs on multiple machines.
- **REST API** — Control Jenkins programmatically.
- **Easy Upgrades** — One-click upgrades from the UI.

■ Tip: Interviewers often ask you to name 5-6 features. Memorize these bullet points!

Q3 How do you start Jenkins?

■ Beginner

Jenkins is distributed as a WAR (Web Application Archive) file. You can start it using the Java command line:

```
# Step 1: Open Command Prompt / Terminal

# Step 2: Go to the folder where jenkins.war is saved

cd C:\Jenkins

# Step 3: Run the command

java -jar jenkins.war

# Jenkins will start on http://localhost:8080 by default

# To use a different port:

java -jar jenkins.war --httpPort=9090
```

After starting, open your browser and go to **http://localhost:8080**. On first run, it will ask for an initial admin password found at: `/var/jenkins_home/secrets/initialAdminPassword`

■ Tip: On Linux, you can also use: `sudo systemctl start jenkins` (if installed via package manager)

Q4 What is Continuous Integration (CI)?

■ Beginner

Continuous Integration (CI) is a software development practice where developers **frequently merge their code changes** into a shared repository — multiple times a day.

Every time code is pushed, an **automated build and test process** runs immediately. This catches bugs early before they pile up.

Example without CI: 5 developers work separately for 2 weeks, then merge all at once → conflicts everywhere, hard to find bugs.

Example with CI: Each developer pushes code daily. Jenkins builds and tests automatically → bugs caught the same day they are introduced.

- Detects bugs early in the development cycle
- Reduces integration problems (no 'big bang' merges)
- Always keeps a working build available
- Provides instant feedback to developers

■ Tip: CI = merge often + build automatically + test automatically

Q5 What is a CI/CD Pipeline?

■ Beginner

A CI/CD Pipeline is a series of automated steps that your code goes through — from a developer writing it to it being live in production. It is the **backbone of DevOps**.

CI (Continuous Integration) = Merge code → Build → Run Tests

CD (Continuous Delivery) = Automatically prepare code for release

CD (Continuous Deployment) = Automatically push to production

A typical pipeline looks like this:

```
Code Push → Build → Unit Tests → Code Analysis  
→ Integration Tests → Package → Deploy to Staging  
→ UAT Tests → Deploy to Production ■
```

- **Build Stage:** Compile the code
- **Test Stage:** Run automated tests
- **Deploy Stage:** Push to servers
- **Monitor Stage:** Check health after deploy

■ Tip: Think of a pipeline like an assembly line in a factory. Each stage does one job and passes work to the next.

Q6 What is a Jenkins Pipeline?

■ Beginner

A Jenkins Pipeline is a **set of automated steps** (called stages) that define how your code moves from source to production. It is written as code using a **Groovy-based DSL** (Domain Specific Language).

Jenkins Pipelines are defined in a file called **Jenkinsfile** which lives in your code repository.

```
// Simple Jenkinsfile example
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo 'Compiling the code...'
        sh 'mvn compile'
      }
    }
    stage('Test') {
      steps {
        echo 'Running tests...'
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      steps {
        echo 'Deploying to server...'
        sh './deploy.sh'
      }
    }
  }
}
```

Each **stage** is a phase (Build, Test, Deploy). Each **step** is a command inside that stage.

■ Tip: Pipelines as code (Jenkinsfile) are better than UI-configured jobs because they can be version-controlled in Git!

Q7 What are the 3 types of Jenkins Pipelines?

■ Intermediate

- **1. CI/CD Pipeline** — The complete flow from code commit to deployment. Most commonly referred to when people say 'Jenkins Pipeline'.
- **2. Declarative Pipeline** — A newer, simpler syntax with strict structure. Easier to read and write. Recommended for beginners.
- **3. Scripted Pipeline** — Older Groovy-based syntax. More flexible and powerful but harder to read.

Declarative Pipeline example (simple, structured):

```
pipeline {
  agent any
  stages {
    stage('Test') {
```

```
steps {  
  sh 'mvn test'  
}  
}  
}
```

Scripted Pipeline example (flexible, complex):

```
node {  
  stage('Test') {  
    sh 'mvn test'  
  }  
}
```

■ Tip: Always prefer Declarative Pipeline in interviews unless asked about scripted. Declarative is the modern standard.

Q8 What is a Jenkinsfile?

■ Intermediate

A Jenkinsfile is a **text file** that contains the entire pipeline definition written in Groovy. It is stored in the **root of your code repository** (in Git), just like your source code.

- **Version Controlled** — Stored in Git, so every change is tracked
- **Code Review** — Team members can review and approve pipeline changes
- **Audit Trail** — You can see who changed the pipeline and when
- **Reusable** — Same Jenkinsfile can be used across branches and projects

Without Jenkinsfile → Pipeline configured manually in Jenkins UI (hard to track changes).

With Jenkinsfile → Pipeline is code, stored in Git, reviewable, repeatable ■

■ Tip: Storing pipeline as code in a Jenkinsfile is called 'Pipeline as Code' — a DevOps best practice.

Q9 What is Groovy in Jenkins?

■ Intermediate

Groovy is a **dynamic programming language** that runs on the Java platform. Jenkins uses Groovy as the scripting language for writing pipelines.

You don't need to be a Groovy expert to use Jenkins — the Declarative Pipeline syntax is very simple. But for advanced pipelines, Groovy knowledge helps.

```
// Groovy in Jenkinsfile – defining variables and using logic
pipeline {
  agent any
  environment {
    APP_NAME = 'my-app'
    VERSION = '1.0.0'
  }
  stages {
    stage('Info') {
      steps {
        // Using Groovy string interpolation
        echo "Building ${APP_NAME} version ${VERSION}"

        // Groovy if/else in script block
        script {
          if (env.BRANCH_NAME == 'main') {
            echo 'This is the main branch!'
          } else {
            echo 'This is a feature branch'
          }
        }
      }
    }
  }
}
```

```
}  
  
}  
  
}  
  
}  
  
}
```

- Groovy syntax is similar to Java — easy for Java developers
- Supports closures, dynamic typing, and scripting features
- Used in both Scripted and Declarative pipelines

■ Tip: Groovy is to Jenkins what JavaScript is to web browsers — it's the scripting power behind the tool.

Q1
0

What is Jenkins DSL?

■ Beginner

DSL stands for **Domain Specific Language**. Jenkins Job DSL is a **plugin** that lets you define and create Jenkins jobs using Groovy code instead of clicking through the Jenkins UI.

Without DSL: You manually create 50 Jenkins jobs one by one through the web interface — slow and error-prone.

With Job DSL: Write code to generate all 50 jobs automatically — fast and repeatable!

```
// Job DSL example: Creates a build job automatically  
job('my-project-build') {  
  description('Builds the my-project application')  
  
  scm {  
    git('https://github.com/company/my-project.git')  
  }  
  
  triggers {  
    scm('H/15 * * * *') // Poll every 15 mins  
  }  
  
  steps {  
    maven('-e clean install')  
  }  
  
  publishers {  
    mailer('team@company.com')  
  }  
}
```

■ Tip: Job DSL is about creating jobs as code. Jenkinsfile is about defining pipeline steps. They are different things!

Jobs, Setup & Configuration

Q1 1 How do you set up a Jenkins Job?

■ Beginner

A Jenkins **Job** (also called a Project) is a unit of work — like 'build my app' or 'run my tests'. Here are the steps to create one:

- **Step 1:** Click '**New Item**' from the Jenkins dashboard
- **Step 2:** Enter a name for your job (e.g., 'my-app-build')
- **Step 3:** Choose the job type — 'Freestyle Project' for simple jobs, 'Pipeline' for complex ones
- **Step 4:** Click **OK**
- **Step 5:** Configure the job — set Source Code Management (Git URL), Build Triggers (when to run), Build Steps (what to do), Post-build actions (what to do after)

Types of Jenkins Jobs:

- **Freestyle Project** — Simple, configurable via UI, good for beginners
- **Pipeline** — Complex workflows, defined with Jenkinsfile
- **Multibranch Pipeline** — Automatically creates jobs for each Git branch
- **Folder** — Organizes multiple jobs together

■ Tip: For interviews, know the steps to create a Freestyle project AND a Pipeline project.

Q1 2 What are the requirements to use Jenkins?

■ Beginner

To use Jenkins for CI/CD, you need these basic components:

- **Java** — Jenkins runs on Java (JDK 11 or 17 recommended)
- **A Source Code Repository** — Like GitHub, GitLab, or Bitbucket where your code lives
- **A Build Script** — Like Maven (pom.xml), Gradle (build.gradle), or npm (package.json) to build your project
- **Jenkins Server** — The machine where Jenkins is installed
- **Sufficient RAM** — Minimum 256MB, recommended 1GB+

Jenkins integrates most commonly with:

- **Version Control:** Git, SVN (Subversion), Mercurial
- **Build Tools:** Maven, Gradle, Ant, npm
- **Test Frameworks:** JUnit, TestNG, Selenium
- **Containers:** Docker, Kubernetes

■ Tip: In most real projects: Jenkins + Git + Maven (for Java) or Jenkins + Git + npm (for Node.js) is the common setup.

Q1 3 What are some important Jenkins Environment Variables?

■ Beginner

Jenkins automatically provides built-in environment variables that you can use in your pipeline scripts. These are very useful!

```
// Using environment variables in Jenkinsfile
pipeline {
```

```

agent any
stages {
  stage('Info') {
    steps {
      echo "Job Name: ${env.JOB_NAME}"
      echo "Build Number: ${env.BUILD_NUMBER}"
      echo "Workspace: ${env.WORKSPACE}"
      echo "Build URL: ${env.BUILD_URL}"
      echo "Branch: ${env.BRANCH_NAME}"
    }
  }
}

```

- **\$JOB_NAME** — Name of the Jenkins job (e.g., 'my-app-build')
- **\$BUILD_NUMBER** — Current build number (e.g., 42)
- **\$WORKSPACE** — Path to the job's workspace folder
- **\$BUILD_URL** — URL to see this build's results
- **\$JENKINS_URL** — URL of the Jenkins server
- **\$NODE_NAME** — Name of the agent/node running the build
- **\$BRANCH_NAME** — Git branch name (in multibranch pipelines)

■ Tip: You can see all available env variables at: <http://your-jenkins-url/env-vars.html/>

Q1
4

How do you configure a Third-Party Tool in Jenkins?

■ Beginner

Jenkins works with many external tools (Maven, Docker, SonarQube, etc.) through plugins. Here's the 4-step process:

- **Step 1: Install the tool** — Install Maven/Docker/etc. on the Jenkins server machine
- **Step 2: Install the Jenkins plugin** — Go to Manage Jenkins → Plugins → Available → Search and install the relevant plugin (e.g., 'Maven Integration Plugin')
- **Step 3: Configure the tool** — Go to Manage Jenkins → Global Tool Configuration → Set the tool path (e.g., Maven home directory)
- **Step 4: Use in Pipeline** — Now use the tool in your Jenkinsfile

```

// Using Maven after configuration
pipeline {
  agent any
  tools {
    maven 'Maven-3.8.6' // Name set in Global Tool Config
  }
  stages {
    stage('Build') {
      steps {

```

```
sh 'mvn clean package'
```

```
}
```

```
}
```

```
}
```

```
}
```

■ Tip: The most common tools to configure: Maven, JDK, Git, Node.js, Docker.

Plugins & Integrations

Q1
5**What are Jenkins Plugins? Name some important ones.**

■ Intermediate

Plugins are **add-ons that extend Jenkins functionality**. Jenkins by itself is basic — plugins are what make it powerful. There are 1800+ plugins available.

Install plugins: Manage Jenkins → Manage Plugins → Available tab

Essential Plugins to Know:

- **Git Plugin** — Connects Jenkins with Git repositories (GitHub, GitLab)
- **Maven Integration** — Build Java projects with Maven
- **Pipeline Plugin** — Enables Jenkinsfile-based pipelines
- **Blue Ocean** — Modern, visual UI for Jenkins pipelines
- **Docker Plugin** — Build and run Docker containers
- **Kubernetes Plugin** — Run Jenkins agents on Kubernetes pods
- **SonarQube Scanner** — Code quality analysis
- **Slack Notification** — Send build results to Slack
- **Email Extension** — Send detailed email notifications
- **JUnit Plugin** — Display test results in Jenkins UI
- **Credentials Plugin** — Securely store passwords and API keys

■ Tip: Interviewers may ask: 'Which plugins have you used?' Know at least 5-6 plugins with their purpose.

Q1
6**Why is Jenkins used with Selenium?**

■ Intermediate

Selenium is a popular tool for **automated UI testing** of web applications. When combined with Jenkins, your UI tests run automatically as part of the CI/CD pipeline.

How it works:

- Developer pushes new code to Git
- Jenkins detects the change and triggers a build
- After build succeeds, Jenkins runs the Selenium test suite automatically
- Selenium opens a browser, clicks buttons, fills forms, and verifies results
- Jenkins collects the results and shows pass/fail report
- Team gets notified if any UI test fails

```
// Jenkinsfile running Selenium tests
stage('UI Tests') {
  steps {
    sh 'mvn test -Dtest=SeleniumTestSuite'
  }
  post {
    always {
      junit 'target/surefire-reports/*.xml'
```

```
}  
}  
}
```

Benefits: UI bugs are caught automatically without manual testing after every code change.

■ Tip: This is a very common interview question for QA automation and DevOps roles!

Q1 7 How does Jenkins integrate with Kubernetes?

■ Intermediate

Kubernetes is a container orchestration platform. Jenkins integrates with Kubernetes to **run build agents (workers) as pods** — creating them on demand and destroying them when the build finishes.

Why integrate Jenkins with Kubernetes?

- No need to maintain permanent build agents — pods spin up and down automatically
- Better resource usage — agents only use resources when a build is running
- Easy scaling — if 20 builds run simultaneously, 20 pods spin up
- Fault tolerance — if a pod fails, Kubernetes restarts it

How to integrate:

- Install the **Kubernetes plugin** in Jenkins
- Configure the Kubernetes cluster credentials in Jenkins
- Define a pod template in your Jenkinsfile
- Each stage of the pipeline runs inside a Docker container in the pod

```
// Jenkinsfile using Kubernetes agent  
  
pipeline {  
  agent {  
    kubernetes {  
      yaml '''  
      apiVersion: v1  
      kind: Pod  
      spec:  
        containers:  
        - name: maven  
          image: maven:3.8.6  
          command: ["cat"]  
          tty: true  
      '''  
    }  
  }  
  stages {  
    stage('Build') {  
      steps {  
        container('maven') {
```

```
sh 'mvn package'
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

■ Tip: Kubernetes + Jenkins is a very popular DevOps stack. Even basic knowledge here impresses interviewers.

How do you create a backup of Jenkins?

■ Beginner

Jenkins stores **everything** in its home directory — all jobs, configurations, build history, plugins, and logs. Backing up Jenkins is as simple as copying this folder.

```
# Jenkins home directory locations:
# Linux: /var/lib/jenkins/
# Windows: C:\Users\...\jenkins\
# Docker: /var/jenkins_home/

# Simple backup command (Linux):
cp -r /var/lib/jenkins/ /backup/jenkins-backup-$(date +%Y%m%d)/

# Or using rsync:
rsync -avz /var/lib/jenkins/ /backup/jenkins/
```

- **\$JENKINS_HOME/jobs/** — All job configurations and build history
- **\$JENKINS_HOME/plugins/** — All installed plugins
- **\$JENKINS_HOME/config.xml** — Main Jenkins configuration
- **\$JENKINS_HOME/secrets/** — Credentials (back this up carefully!)

To **copy/clone a specific job**: Copy the folder at `$JENKINS_HOME/jobs/[job-name]/` to a new folder with a different name, then reload Jenkins.

■ Tip: Use the ThinBackup plugin for scheduled automated backups. Much easier than manual copying!

How do you deploy a custom build of a Core Plugin?

■ Beginner

Sometimes you need to use a modified (custom-built) version of a Jenkins plugin instead of the official one. Here are the steps:

- **Step 1:** Stop Jenkins completely
- **Step 2:** Copy your custom .hpi file to `$JENKINS_HOME/plugins/` directory
- **Step 3:** Delete the old expanded plugin directory (folder with same name as plugin)
- **Step 4:** Create an empty file called `<plugin-name>.hpi.pinned` (this prevents Jenkins from auto-updating and overwriting your custom version)
- **Step 5:** Start Jenkins again

```
# Example: Deploying custom 'git' plugin

# 1. Stop Jenkins
sudo systemctl stop jenkins

# 2. Copy custom .hpi file
cp my-custom-git.hpi /var/lib/jenkins/plugins/git.hpi
```

```
# 3. Remove expanded old plugin directory
rm -rf /var/lib/jenkins/plugins/git/

# 4. Create pinned file (prevents auto-update)
touch /var/lib/jenkins/plugins/git.hpi.pinned

# 5. Start Jenkins
sudo systemctl start jenkins
```

■ Tip: The .hpi.pinned file is the key! Without it, Jenkins may overwrite your custom plugin on next update.

Q2
0

How do you move or copy Jenkins from one server to another?

■ Intermediate

Moving Jenkins to a new server is straightforward — it's basically moving the Jenkins home directory:

- **Method 1 (Copy entire Jenkins):** Zip and transfer the entire \$JENKINS_HOME directory to the new server. Install same Java version, install Jenkins, replace home directory, start Jenkins.
- **Method 2 (Move one job):** Copy just the \$JENKINS_HOME/jobs/[job-name]/ folder to the same path on the new server.
- **Method 3 (Clone a job):** Copy a job folder and give it a different name. Jenkins will treat it as a new job.
- **Method 4 (Rename a job):** Rename the job's folder in \$JENKINS_HOME/jobs/. Then restart Jenkins.

```
# Moving entire Jenkins to new server

# On OLD server:
tar -czf jenkins_backup.tar.gz /var/lib/jenkins/
scp jenkins_backup.tar.gz user@new-server:/tmp/

# On NEW server (after installing Jenkins):
sudo systemctl stop jenkins
sudo tar -xzf /tmp/jenkins_backup.tar.gz -C /
sudo systemctl start jenkins
```

■ Note: Make sure the new server has the same Java version and the same OS architecture for best compatibility.

Q2
1

A build is failing in Jenkins — how do you debug it?

■ Intermediate

Build failures happen. Here's a **systematic approach** to debug:

- **Step 1: Check Console Output** — Click on the failed build → Click 'Console Output'. Read the error message carefully. 90% of the time the error is explained here.
- **Step 2: Check recent code changes** — Use 'Changes' tab to see what files changed in this build. Did someone check in broken code?
- **Step 3: Check environment** — Did any server, database, or dependency go down? Check if other jobs are also failing.
- **Step 4: Reproduce locally** — Pull the same code and run the same build command on your local machine. Does it fail there too?
- **Step 5: Check build logs** — Look at the full log for warnings that appeared before the error.
- **Step 6: Fix and re-run** — After fixing, click 'Build Now' to verify the fix works.

Common causes of build failures:

- Compilation error — someone pushed code that doesn't compile
- Test failure — a unit test is failing
- Missing dependency — a library or service is unavailable
- Environment issue — wrong Java version, missing environment variable
- Disk full — the Jenkins server has no free disk space

■ Tip: Always start with Console Output — it's your best friend when debugging Jenkins failures.

Q2
2

What happens if a pipeline stage fails? (Restart from Stage)

■ Beginner

If a stage in your pipeline fails, you **don't have to restart the entire pipeline from the beginning**. Jenkins has a feature called '**Restart from Stage**'.

Example scenario:

- Your pipeline has 5 stages: Build → Test → Package → Deploy-Staging → Deploy-Production
- Stages 1-3 completed successfully (took 20 minutes)
- Stage 4 (Deploy-Staging) failed because the staging server was temporarily down
- The server is now back up

Instead of running all 5 stages again (wasting 20 minutes), you click **Restart from Stage** and pick Stage 4. Jenkins will run only from Stage 4 onwards.

How to use: Go to the failed build → Click on the failed stage → Select 'Restart from Stage'

```
// To enable restart from stage, use the  
// Declarative Pipeline syntax (not Scripted)  
// Jenkins automatically supports this feature  
// in Declarative Pipelines ■
```

■ Tip: Restart from Stage saves a lot of time in long pipelines. Mention this in interviews to show practical knowledge!

Explain Jenkins Master-Agent (Master-Slave) Architecture

■ Intermediate

Jenkins uses a **Master-Agent architecture** to distribute work across multiple machines. This allows Jenkins to run many builds simultaneously and on different environments.

Jenkins Master:

- The main Jenkins server
- Manages the web UI and configuration
- Schedules jobs and assigns them to agents
- Monitors agent status and collects results
- Stores all job configurations and build history

Jenkins Agent (formerly called Slave):

- A worker machine that runs the actual build
- Receives instructions from the master
- Can run on Linux, Windows, Mac, or Docker
- Multiple agents can run builds in parallel

```
// Assign a job to a specific agent in Jenkinsfile
pipeline {
  agent { label 'linux-agent' } // Run on agent labeled 'linux-agent'

  stages {
    stage('Build') {
      agent { label 'windows-agent' } // Override for this stage
      steps {
        bat 'build.bat' // Windows command
      }
    }
  }
}
```

Why use multiple agents? Run builds faster (parallel), test on different OS versions, separate workloads.

■ Tip: In interviews, draw this on paper: [Master] --assigns jobs--> [Agent1, Agent2, Agent3]

How does Jenkins work internally? (End-to-End Flow)

■ Intermediate

Here's the complete flow of how Jenkins works when a developer pushes code:

- **1. Developer pushes code** to Git repository
- **2. Jenkins detects the change** — either by polling the repo every few minutes (SCM polling) or via a webhook (Git notifies Jenkins instantly)
- **3. Jenkins triggers a build** — starts a new build run

- **4. Jenkins allocates an agent** — assigns the job to an available agent
- **5. Agent checks out code** — downloads the latest code from Git
- **6. Pipeline stages run sequentially** — Build → Test → Package → Deploy
- **7. Stage results are reported back** to the master
- **8. If a stage fails** — pipeline stops, notification sent (email/Slack)
- **9. If all stages pass** — code is deployed to server
- **10. Notification sent to team** — success/failure summary

```
// Trigger types in Jenkinsfile
pipeline {
  triggers {
    // Option 1: Poll SCM every 5 minutes
    pollSCM('H/5 * * * *')

    // Option 2: Webhook (triggered by Git push)
    // Configure webhook in GitHub settings
    // URL: http://jenkins-url/github-webhook/
  }
}
```

■ **Tip:** For interviews, be able to explain this flow clearly. It shows you understand the big picture of CI/CD.

Q2
5

What are some alternatives to Jenkins?

■ **Intermediate**

While Jenkins is the most popular, there are several other CI/CD tools available. Knowing them shows broad DevOps awareness:

- **GitHub Actions** — Built into GitHub, very popular for open-source projects, YAML-based
- **GitLab CI/CD** — Built into GitLab, powerful and easy to use for GitLab users
- **CircleCI** — Cloud-based, fast, easy setup, popular for startups
- **TeamCity** — By JetBrains, enterprise-grade, great IDE integration
- **Travis CI** — Cloud-based, was very popular for open-source (GitHub) projects
- **Bamboo** — By Atlassian, integrates well with Jira and Bitbucket
- **Azure DevOps Pipelines** — Microsoft's cloud CI/CD, excellent for Azure deployments
- **GoCD** — Open-source, great for complex deployment pipelines

■ **Note:** Jenkins advantage: Self-hosted (data stays with you), massive plugin ecosystem, highly customizable. Disadvantage: Needs maintenance, older UI.

Q2
6

How would you secure Jenkins?

■ Advanced

Security is critical for Jenkins since it can deploy code to production. Here are key security practices:

- **Enable Authentication** — Turn on login requirement. Use Jenkins' own user database or connect to LDAP/Active Directory.
- **Role-Based Access Control (RBAC)** — Use the 'Role Strategy Plugin'. Give developers only job-level access, not admin access.
- **Use Credentials Plugin** — Never hardcode passwords in Jenkinsfile. Store them in Jenkins Credentials and reference by ID.
- **Enable HTTPS** — Put Jenkins behind a reverse proxy (Nginx) with SSL/TLS.
- **Audit Trail Plugin** — Track who did what and when in Jenkins.
- **Disable CLI** — Disable the Jenkins CLI if not needed (potential attack vector).
- **Keep Jenkins Updated** — Apply security patches regularly.

```
// Using stored credentials in Jenkinsfile (SECURE)
pipeline {
  agent any
  environment {
    // Reference credential by ID, not hardcoded password!
    DB_PASSWORD = credentials('database-prod-password')
    AWS_KEY = credentials('aws-access-key')
  }
  stages {
    stage('Deploy') {
      steps {
        sh 'deploy.sh --password $DB_PASSWORD'
      }
    }
  }
}
```

■ Tip: NEVER put passwords, API keys, or secrets directly in a Jenkinsfile. Always use Jenkins Credentials!

Q2
7

How do you implement Parallel stages in Jenkins?

■ Advanced

Running stages in parallel can greatly speed up your pipeline. For example, running unit tests, integration tests, and code analysis simultaneously instead of one after another.

```
pipeline {
  agent any
```

```

stages {
  stage('Build') {
    steps {
      sh 'mvn compile'
    }
  }

  // Run these 3 stages at the SAME TIME
  stage('Parallel Testing') {
    parallel {
      stage('Unit Tests') {
        steps {
          sh 'mvn test -Dtest=UnitTests'
        }
      }

      stage('Integration Tests') {
        steps {
          sh 'mvn test -Dtest=IntegrationTests'
        }
      }

      stage('Code Analysis') {
        steps {
          sh 'mvn sonar:sonar'
        }
      }
    }
  }

  stage('Deploy') {
    steps {
      sh './deploy.sh'
    }
  }
}

```

Before parallel: 30 min (10+10+10 sequential). After parallel: ~12 min (all 3 run together).

■ Tip: Parallel stages can drastically reduce pipeline time. This shows you understand advanced Jenkins!

Q2
8

How do you handle Jenkins pipeline notifications?

■ Advanced

Keeping the team informed about build results is essential. Jenkins supports multiple notification methods:

```

pipeline {
  agent any

  stages {
    stage('Build') {
      steps { sh 'mvn package' }
    }
  }

  post {
    success {
      // Send Slack message on success
      slackSend channel: '#devops',
        color: 'good',
        message: "■ Build PASSED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    }
    failure {
      // Send email on failure
      emailExt (
        subject: "■ Build FAILED: ${env.JOB_NAME}",
        body: "Build ${env.BUILD_NUMBER} failed. Check: ${env.BUILD_URL}",
        to: 'team@company.com'
      )
    }
    always {
      // Always publish test results
      junit 'target/surefire-reports/*.xml'
    }
  }
}

```

- **post { success {} }** — Runs only when all stages pass
- **post { failure {} }** — Runs only when a stage fails
- **post { always {} }** — Runs regardless of result
- **post { unstable {} }** — Runs when tests fail but build succeeds

■ Tip: The 'post' block in Declarative Pipeline is the standard way to handle notifications and cleanup.

Term	One-Line Definition
Jenkins	Open-source CI/CD automation server written in Java
CI (Continuous Integration)	Auto build & test on every code push
CD (Continuous Delivery)	Auto prepare release; manual deploy trigger
CD (Continuous Deployment)	Auto build, test, AND deploy to production
Pipeline	Automated series of steps: Build → Test → Deploy
Jenkinsfile	Pipeline definition stored as code in Git repository
Stage	A logical phase in the pipeline (e.g., 'Test')
Step	A single command inside a stage (e.g., 'sh mvn test')
Agent/Node	Machine that runs the build job
Master	Main Jenkins server that manages everything
Plugin	Add-on that extends Jenkins functionality
Groovy	Scripting language used to write Jenkins pipelines
DSL	Code-based way to create Jenkins jobs automatically
SCM	Source Code Management (Git, SVN, etc.)
Webhook	Git notifies Jenkins instantly when code is pushed
Build Trigger	What starts a build (code push, schedule, manual)
Workspace	Folder on agent where code is checked out for building
Artifact	Output file of a build (e.g., .jar, .war, .zip)
Blue Ocean	Modern visual UI plugin for Jenkins pipelines

■ Top 10 Most Asked Jenkins Interview Questions:

1. What is Jenkins?
2. What is CI/CD?
3. What is a Jenkins Pipeline?
4. Declarative vs Scripted Pipeline
5. What is Jenkinsfile?
6. What is a Master-Agent architecture?
7. How to handle a failed build?
8. Name 5 Jenkins plugins
9. What is Groovy in Jenkins?
10. How to backup Jenkins?